TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN — BASES DE DATOS II (UTN FRM)

Trabajo Práctico · Unidad 7: NoSQL y Nuevas Formas de Almacenamiento

Proyecto: Food Store | Motores: PostgreSQL 18 & MongoDB 7+ | Alumno: Fabrizio Simon | Fecha: 10/11/2026

1. Modelo Documental y Estrategia de Migración (Parte 1)

Subconjunto seleccionado de Food Store: Se tomó el módulo central de pedidos y ventas compuesto por las tablas relacionales pedido, pedido_producto, cliente, producto y categoria. En PostgreSQL, consultar un pedido implica vincular claves foráneas en cascada.

Esquema documental en MongoDB (foodstore_nosql.pedidos): Se adoptó el patrón de diseño orientado a agregados (*Aggregate-Oriented Design*), encapsulando el ciclo de vida completo del pedido en un único documento:

```
{
  "_id": 2,  // Se conserva id_pedido de PostgreSQL
  "fecha": ISODate("2025-10-30T18:44:05.346Z"),
  "forma_pago": "EFECTIVO",
  "estado": "ENTREGADO",
  "cliente": { "id_cliente": 1, "nombre": "Juan", "email": "juan@test.com" },
  "lineas": [
    { "id_producto": 3, "nombre": "Producto masivo 3", "categoria": "Pizzas", "cantidad": 1, "precio_unitario": 500.00, "subtotal":
    { "id_producto": 4, "nombre": "Producto masivo 4", "categoria": "Categoria masiva 1", "cantidad": 2, "precio_unitario": 600.00,
  ],
  "total": 1700.00
}
```

Justificación técnica de diseño (Embeber vs Referenciar):

• **Líneas de pedido embebidas (1:N fuerte):** Las líneas de detalle carecen de sentido de negocio fuera del pedido. Siempre se consultan juntas para emitir comprobantes o preparar despachos. Embeberlas permite recuperarlas en un único acceso a disco (*single-document read*), sin latencia de JOINs, y garantiza atomicidad nativa en la inserción sin sobrecarga de transacciones distribuidas.

• **Cliente parcialmente embebido (Snapshot histórico):** Se embeben únicamente los atributos esenciales (nombre, email). Esto desacopla el pedido de cambios futuros en los datos maestros del cliente, congelando el comprobante de venta tal como ocurrió en ese instante.

• **Reglas de integridad conservadas:** Se preserva que cantidad > 0 y precio_unitario >= 0 en cada línea; el total del documento equivale exactamente a la suma de los subtotales; y los precios unitarios son históricos e inmutables.

2. Consultas Ejecutadas y Evidencia de Resultados (Parte 2)

Se migraron 6 pedidos reales de Food Store (IDs del 2 al 7) con múltiples productos y distintos estados (ENTREGADO, PENDIENTE, CANCELADO). A continuación se contrastan las 4 consultas ejecutadas en ambos motores:

Objetivo de Consulta	Sintaxis en MongoDB	Sintaxis en PostgreSQL	Resultado Obtenido
4.a) Todos los pedidos con ID y estado	db.pedidos.find({}, {_id: 1, estado: 1})	SELECT id_pedido, estado FROM pedido WHERE id_pedido BETWEEN 2 AND 7	**6 pedidos recuperados (IDs 2, 3, 4, 5, 6, 7). Estados idénticos.**
4.b) Filtrar por estado 'ENTREGADO'	db.pedidos.find({estado: 'ENTREGADO'})	SELECT p.id_pedido, ... WHERE p.estado = 'ENTREGADO'	**3 pedidos: IDs 2 ($1700), 4 ($2050), 7 ($500).**
4.c) Buscar pedidos con producto ID 3	db.pedidos.find({'lineas.id_producto': 3})	SELECT DISTINCT ... JOIN pedido_producto pp WHERE pp.id_producto = 3	**3 pedidos: IDs 2, 4, 6 (coincidencia exacta).**
4.d) Pedidos con total > $2000.00	db.pedidos.find({total: {$gt: 2000}})	SELECT ... GROUP BY p.id_pedido HAVING SUM(...) > 2000	**2 pedidos: IDs 4 ($2050) y 6 ($3500). Umbral segmenta compras mayoristas.**

3. Tabla de Verificación Cruzada entre Motores (Consigna 5)

Consulta / Filtro Evaluado	Cantidad en PostgreSQL	Cantidad en MongoDB	¿Coinciden los IDs?	Detalle de IDs Recuperados
Estado seleccionado ('ENTREGADO')	**3**	**3**	**Sí**	**IDs: [2, 4, 7]**
Producto seleccionado (ID = 3)	**3**	**3**	**Sí**	**IDs: [2, 4, 6]**

Análisis de consistencia: No se registró ninguna discrepancia. Al modelar el ID original como _id numérico e incorporar los precios históricos exactos, las consultas por estado y por producto en arreglo arrojaron conjuntos idénticos en ambos motores.

4. Evaluación Tecnológica y Contraste Crítico con la IA (Parte 3)

Se consultó a una herramienta de IA para comparar PostgreSQL y MongoDB en pedidos de Food Store, analizando accesos frecuentes a pedidos completos y necesidad de coherencia en importes y stock. Se contrastaron críticamente dos afirmaciones de la IA:

Contraste 1 — Transacciones ACID en MongoDB:
• *Afirmación de la IA:* «MongoDB no es recomendable para pedidos porque no soporta transacciones ACID y pueden quedar pedidos grabados sin stock».
• *Contraste oficial:* **Falso y desactualizado.** El manual oficial de MongoDB (*MongoDB Manual - Transactions*) documenta que desde la versión 4.0 existen transacciones multi-documento con semántica ACID completa (session.startTransaction()). Además, al estar las líneas embebidas dentro del pedido, cualquier inserción o actualización es **atómica por naturaleza a nivel de documento** sin requerir transacciones distribuidas.

Contraste 2 — Rendimiento de los JOINs en PostgreSQL:
• *Afirmación de la IA:* «En PostgreSQL cada consulta de un pedido completo colapsa el rendimiento porque requiere 4 JOINs pesados obligatorios».
• *Contraste de laboratorio:* **Exagerado.** En el laboratorio comprobé con EXPLAIN ANALYZE que un JOIN entre pedido y pedido_producto sobre clave foránea indexada toma **0.08 ms** sobre 200,000 registros. La ventaja real de MongoDB no es el tiempo de búsqueda, sino que entrega el objeto BSON/JSON ensamblado directamente a la API web, ahorrando la sobrecarga de mapeo relacional (ORM).

5. Conclusión Técnica Fundamentada (Consigna 7)

Para Food Store recomiendo adoptar una **arquitectura híbrida políglota** con separación de responsabilidades (patrón CQRS).

En el **módulo transaccional y de facturación**, **PostgreSQL** debe mantenerse como la fuente única de la verdad (*Single Source of Truth*). Criterio técnico 1: **Garantía estricta de integridad referencial y unicidad**: las claves foráneas y restricciones CHECK impiden inconsistencias contables, pedidos huérfanos o ventas de productos inexistentes, algo fundamental para auditoría fiscal.

En el **módulo de consulta de la tienda web y mobile**, **MongoDB** es la solución superior para almacenar los pedidos cerrados y el catálogo de productos. Criterio técnico 2: **Patrón de acceso orientado a agregados y escalabilidad de lectura**: la aplicación web consulta el pedido como un único bloque atómico (datos del cliente, líneas y total). Al estar embebidos en un solo documento BSON, MongoDB resuelve la lectura en un único acceso a disco (findOne), evitando la sobrecarga de ensamblado y permitiendo escalabilidad horizontal (*sharding*) sin degradar la base transaccional principal.